Toolchain, and How a Program Becomes a Program

Keep this. You will want the command table and the warning-reading example again in later weeks — this course does not repeat toolchain explanations once the semester's projects start. (The C++ program shape and the four-stage pipeline are in the Lecture 1 handout.)

1. Key terms

Term	Definition
argc / argv	main's parameters: argc is how many command-line words were typed; argv is a char** — a pointer to an array of C-string pointers, one per word, argv[0] being the program's own name.
Exit code	main's return value. 0 conventionally means success. The shell that launched your program can read it (echo $? right after running something).
Warning vs. error	An error stops the build; a warning does not — but in this course, the strict preset (what grading uses) turns every warning into an error with -Werror.
Debugger	A program that pauses another program mid-execution and lets you inspect its variables and call stack, one line at a time.
Sanitizer (ASan/UBSan) — taught week 4	Extra checks compiled into your program (the debug preset turns these on) that stop execution the instant it does something illegal with memory, and print exactly where.

2. Reading a compiler warning

```
warn.cpp: In function 'int main()':
warn.cpp:2:9: warning: unused variable 'oops' [-Wunused-variable]
    2 |     int oops;
      |         ^~~~
```

- Line 1 — which function the problem is in.
- warn.cpp:2:9 — file, line, column. Go there first.
- [-Wunused-variable] — the warning's name. Every warning category has one; you can search it.
- The last two lines — the actual source line, with a caret (^) under the exact token.

> **THIS COURSE'S POLICY** cmake --preset debug shows warnings but still builds. cmake --preset strict (what grading runs) turns every one into a hard error. Zero warnings under strict, every time, before you submit.

3. What debugging actually means

- “Bug” predates computers by decades (Edison, 1878). The famous Harvard Mark II moth (Sept. 9, 1947) didn't coin the word — it's just the most-documented literal case of it.
- Compile-time error: the compiler catches it and refuses to build. You get a message, not a program. Runtime error: the build succeeds, but the program crashes, hangs, or quietly computes a wrong answer. The compiler cannot see this — it's what a debugger is for.

> **DEBUGGING, PROPERLY** Not “clear the complaints top to bottom.” Notice the wrong behavior → form a hypothesis → test it → fix the actual cause → re-check the WHOLE program. Zero warnings and “correct” are not the same claim.

4. Breakpoints and stepping in VS Code

- Click the gutter left of a line number (or press F9) to set a breakpoint. Press F5 to launch — execution runs at full speed until it hits the breakpoint, then freezes.

Key	Action
F9	Toggle a breakpoint on the current line
F5	Launch / Continue to the next breakpoint
F10	Step Over -- run this line, skip into any calls
F11	Step Into -- follow execution into a function call
Shift + F11	Step Out -- finish this function, return to caller

5. Two examples: variables, console I/O, and a mystery

```
int items;
double price;
std::cout << "Items: "; std::cin >> items;
std::cout << "Price each: "; std::cin >> price;
double total = items * price;
std::cout << "Total: $" << total;
```

- Same pattern, on doubles — run it and it prints “NOT equal,” every time (why: next week).

```
double a = 0.1, b = 0.2;
if (a + b == 0.3) std::cout << "equal";
else std::cout << "NOT equal";   // always prints this
```

6. Command reference

Command	What it does

cmake --preset debug	Configure the debug build (sanitizers on). Use this for all coursework.
cmake --build --preset debug	Compile everything.
ctest --preset debug	Run the test suite and print a pass/fail summary.
./build/debug/bin/game	Run the Lab 0 demo.
echo $?	Print the exit code of the last command you ran.
cmake --preset strict && cmake --build --preset strict && ctest --preset strict	Exactly what the grader runs. Run this yourself before every submission.

7. Your VS Code environment, and Cline

Extension	What it's for
Remote - WSL	Connects VS Code (Windows) to the compiler (Linux/WSL2). Bottom-left corner should say WSL: Debian.
C/C++ + CMake Tools	Language support, and the debug/strict/release preset switcher in the status bar.
GDB (via C/C++ extension)	Powers the F5 debugger from today's lecture and Lab 0.
Cline	This course's AI assistant, backed by DeepSeek via your own API key (BYOK). Installed this week; not used until week 4 — see the callout below.

> **DO NOT USE IT YET** Cline is installed in Lab 0 setup so it is ready when the policy allows it. Weeks 1–3 are AI-off, no exceptions — checked by closed-book in-lab exercises. Your DeepSeek key goes into Cline's own settings panel, never into any file in the repo.

8. Why this week matters for everything after it

Nothing here is examined on its own, but every later week assumes it:

- When you see “undefined reference” in week 4 while writing your first multi-file program, you will already know it means a linker problem, not a typo in your code.
- When header guards are formally taught in week 5, you will already know why: without them, the same header text gets pasted twice, and the compiler sees two definitions.
- argv, the char** you used today without being told it was a pointer, is exactly the kind of value pointer arithmetic operates on in weeks 7–8.
- Every single project this semester is graded under the strict preset. The warning-reading skill from today is not optional politeness — it is how you avoid a build failure at 11:58pm before a deadline.

> **TODAY'S LAB** Lab 0 puts a face on everything above: a green build, passing tests, a debugger

> breakpoint, and a sanitizer report you cause on purpose. See the separate Lab 0 handout for the four tasks.